Analyzing the Effect of Network Width on Graph Transformers

Progress report — what was done, what came out, and what it means

Weeks 1–5 of the department calendar · 2026-09-24 → 2026-10-21 · raw log: `docs/CHANGELOG.md` · questions: `docs/WIDTH-QUESTIONS.md`

Summary.

1. **Width has to be measured with the learning rate tuned per width.** With one fixed rate, wider models look up to 0.4 worse than they are; the best rate falls as width grows.
2. **Width has a floor and a data-dependent ceiling.** Below a minimum width the model cannot fit at all; above some width it memorizes, and that ceiling moves up with more training data (2000 → 8000 graphs removes it).
3. **Our first benchmark was too easy.** It leaked component membership through node indices and graph statistics. We built an audited replacement (`swap`) on which statistics and short-range checks are at chance.
4. **On the clean task, width decides whether learning starts at all.** At $n = 64$ only $m \geq 64$ leave the trivial predictor; at $n = 128$ nothing up to $m = 128$ does.
5. **The width needed to learn connectivity grows superlinearly in n — about $n^{1.7}$ over $n =$ 32–56 — even with ample data.** With 32 000 graphs, fitting and generalizing come apart ($n^{1.55}$ vs $n^{2.27}$); with 128 000 graphs they coincide, and the shared critical width grows as $n^{1.94}$ at learning rate $3 \cdot 10^{-3}$ and $n^{1.74}$ at 10^{-3} (all four sizes; seed-bootstrap 90 % range 1.59–1.90). Doubling the training steps does not lower it, and neither does a fixed-width input encoding (Q6). Theory says this depth needs far less width just to **represent** connectivity.
6. **Depth saves little width at small n, but much more as n grows.** At $n = 40$, two layers learn connectivity at width $\approx 1.1n$ (45) and six layers at $\approx 0.75n$ (30); beyond 4–6 layers depth buys nothing. A 2-layer model solves graphs of diameter ≈ 10 — it does not follow paths hop by hop. But across sizes the 2-layer critical width grows as $\approx n^3$, not linearly as the constant-depth construction allows: at $n =$ 56 two layers need about 3× the width of six (fit 168 vs 53) and do not generalize to 0.95 by width 192 (Q7c).
7. **Sanford et al.'s experiment reproduces, but cannot test their width claim.** Their GraphQA numbers come out at 93.0 / 98.2 (theirs 92.9 / 98.0), yet a 2-hop classifier scores 0.995 and width 32 already gives 98. With their encoding on the audited data, no width up to 192 generalizes (at 100 000 steps the wide models memorize).
8. **Encoding matters as much as size: Sanford et al.'s edge tokens are far harder to learn than adjacency rows.** With the same all-pairs supervision and budget, no width up to 96 reaches the threshold at any n. With 4× the steps they learn, but at 2.2× the adjacency-row width at $n = 32$ (48 vs 22) and $\geq 2.4\times$ at $n = 40$ — at least 5× Sanford et al.'s $\sqrt{N}$ for this very encoding.
9. **The hops left to compute set the width — and its growth.** Giving each token its 2-hop neighbourhood cuts the critical width 2.3–3.2× and the growth from $n^{1.74}$ to $n^{1.27}$ (over $n =$ 32–72); 4 free hops leave width 4–7 growing as $n^{1.03}$. The standard random-walk encoding (RWSE) changes nothing, and its common 16-step version leaks connectivity.
10. **Inside the model, "connected" is built up a little in every layer — not by doubling the reach, and not stored per node.** Linear probes on the residual stream find no distance hierarchy (3-hop information appears at layer 1 and stays flat; 1-hop detail fades), and a node's own vector does not hold its reachable set (AUC ≤ 0.89). The answer is read from **pairs** of vectors (AUC 0.99–1.00). Models well below the critical width climb at the same pace for a layer or two, then fall behind.

1. Question and set-up

Thesis question. How does the width of a graph transformer — its embedding dimension m — determine what graph problems it can solve and learn, and how does that need scale with graph size n?

Task **connectivity matrix**. Input: a graph on n nodes. Output: the $n \times n$ matrix R with $R_{ij} = 1$ iff i and j are in the same component. Dense per-pair supervision; a single connected/disconnected bit per graph does not train (stalls at $\ln 2$, June results). Task, tokens and read-out are taken from Ye et al. (2026), who study which solution this model learns.

Model transformer encoder whose tokens are the rows of $A + I$ (one token per node), pre-norm blocks, multi-head attention with head dimension fixed at 8 (so heads $= m/8$), pairwise bilinear read-out $HWH^\top$.

Width m, the embedding (residual-stream) dimension. Every part of the model scales with it: attention uses $m/8$ heads of dimension 8 (so heads $\times$ head size $= m$, the product mH that the theory bounds), and each feed-forward layer is $4m$ wide. Depth L = number of blocks.

Metrics **exact-match** (the whole matrix right) and **pair accuracy** (fraction of entries right). The trivial "everything connected" predictor scores 0.5 exact-match (half the graphs are connected) and $\approx$ 0.75–0.77 pair accuracy.

Protocol train / validation / test split; learning rate and checkpoint picked on **validation**, reported on test; 3 seeds unless stated. **Critical width** m^* = the smallest m at which at least 2/3 of seeds reach the threshold (0.90 / 0.95 / 0.99).

Compute sweeps run on Kaggle (2 × T4) as 6–8 parallel shards (`width_sweep.py`, `kaggle/width_sweep.ipynb`); every run is one JSON line in `results/width/`.

2. Why these numbers

Every number in the set-up is either **principled** (follows from the question or a measurement), **checked** (chosen, then tested and shown not to drive the result), **inherited** (the default of the June connectivity code this work started from, kept for continuity), or **budget** (set by Kaggle's 30 GPU-hours a week). The tags below say which.

The model and what "width" changes

Choice	Value	Why
width m	embedding dim	The residual-stream dimension is what every theory bound calls width (the m in mHp) and what "model width" means for transformers. One knob that makes the whole model wider in proportion. **principled**
head size	8	Fixed so that only width moves: a wider model gets **more** heads, not bigger ones. 8 is the largest size that still gives the narrowest model ($m = 8$) one head; the literature sweep flagged that changing head size together with width confounds the two (Q5 would vary them separately). **principled**
heads H	$m/8$	Follows from the fixed head size. Heads × head size = m, so our width equals the product mH in Sanford et al.'s bounds. **principled**
feed-forward	$4m$, ReLU	The standard transformer ratio (Vaswani et al. 2017), kept proportional so it scales with m rather than being a second width. **inherited**
blocks	pre-norm	Pre-norm residual blocks train stably without careful initialisation. **inherited**
input	adjacency row $A + I$	One token per node carrying its neighbour set — the tokenization of Yehudai et al. 2025, so their linear-width result is the comparison point. Q6 replaced it with a fixed 96-wide code and the scaling held. **checked**
read-out	$HWH^\top$	Scores every node pair, giving the $n \times n$ connectivity matrix directly; symmetrised because connectivity is symmetric. **inherited**
dropout, weight decay	0, 0	In the July chain-of-thought experiments weight decay alone blocked circuit formation (it pushes attention towards low rank) and dropout gave no benefit; both were left off here and not re-tested on this task. **inherited**

Training

Choice	Value	Why
optimiser	Adam	Standard for transformers; no weight decay (above). **inherited**
schedule	5 % warm-up, cosine	Q1 showed wide models diverge at high rates without warm-up; Q1b added it and the instability went away. **checked**
learning rate	tuned per width	Q1: one fixed rate fakes a width effect of up to 0.4; the best rate falls as width grows. Grids: $3 \cdot 10^{-4}$ … $3 \cdot 10^{-2}$ in Q1/Q1b, where the best rate fell from 10^{-2}–$3 \cdot 10^{-2}$ at small widths to 10^{-3} at $m = 256$. Q3 used $3 \cdot 10^{-3}$ (the only rate that started learning at $n = 64$) and then re-checked with 10^{-3}, which helped only the widest models at $n = 56$. **checked**
batch size	128	Large enough for stable gradients, small enough for many steps per epoch; never varied. **inherited**
steps	4 800 → 24 000	Held **fixed** across data sizes so more data never means more training. 4 800 is Q1′s 300 epochs × 16 batches; 24 000 is where the learnability probe had converged (it was at 0.95–0.98 halfway through 24 000); 48 000 in the step check moved nothing. **checked**
evaluation	≈ 15 times per run	Enough points for learning curves; training accuracy on the first 8 000 training graphs only, so evaluation stays cheap at 128 000. **budget**
checkpoint	final epoch	Choosing the "best" epoch by validation exact-match picks epoch 1 when exact-match never moves, hiding real learning (the 10-05 correction). The cosine schedule makes the final epoch a fair choice. **checked**

Data

Choice	Value	Why
generator	`swap`	The only one of five candidates on which every heuristic we could build — graph statistics, ≤ 4-hop checks, node indices — sits at chance. **principled**
chords	¼ × blob size	Sparse enough that graphs are not locally solvable, dense enough that diameter grows only $\approx \log n$ (a pure cycle's diameter grows $\approx n/2$, and nothing learned at $n \geq 32$ in pilot 1). **checked**
$d_{\min}$	6 hops	Every cycle the two swapped edges close has ≥ 7 edges, so closed-walk counts up to length 6 cannot see the label. With 5, statistics still predicted the label at $n = 32$ (0.63). Cost: graphs below $n \approx 32$ cannot be built. **checked**
blob split	$n_a \sim U[n/4, 3n/4]$	Avoids tiny blobs that are trivially dense or cannot hold a far-apart pair. **principled**
training graphs	2 000 → 128 000	Raised until the data ceiling disappeared: at 32 000, fitting and generalizing still came apart; at 128 000 they coincide. 2 000 was the June default. **checked**
val / test	400 / 400	Separate seeds; 400 graphs give ≈ 110 000–620 000 scored node pairs each ($n = 24$–56) — far more than needed for pair accuracy. **inherited**

Measurement

Choice	Value	Why
main metric	pair accuracy	Exact-match needs all $\approx n^2/2$ pairs right, so it gets stricter as n grows and would build a fake n-dependence into the result. Pair accuracy is comparable across n. **principled**
threshold	0.95	Well above the trivial ≈ 0.75 and below the ≈ 0.99 the best models reach, so the crossing sits on the steep part of the curve. 0.90 and 0.99 were also reported in Q1–Q3; they shift m^* but not the conclusions. **checked**
m^*	per-seed crossing	Interpolated in $\log_2 m$ between grid widths, then averaged over seeds — sturdier than crossing the averaged curve when one seed dips at a single width. **principled**
seeds	2–3	Three in the early and fine grids; two in the large-data checks, where seeds agreed within $\approx \pm 3$ and runs cost most. **budget**

Sweep grids

Choice	Value	Why
widths	16 … 192	Powers of two, with in-between points (24, 48, 96) near each threshold so the crossing is located to within ≈ 20 %. Later sweeps placed the grid around each size's or depth's expected threshold to spend no runs far from it. **budget**
graph sizes	$n = 32$ … 56	The range where m^* can be measured: `swap` cannot be built below ≈ 32, and at $n \geq 64$ nothing learned within affordable width. $n = 24$ in Q1 was the June default. **budget**
depth	$\lceil \log_2 n \rceil$, fixed 6	Theory's depth for connectivity; fixed at 6 — which is $\lceil \log_2 n \rceil$ for every n in 33–64 — so depth does not step up between sizes. Q7a varied it from 2 to 8. **principled**
projected input	$k = 96$	The smallest code width that keeps each neighbour set recoverable (≈ 99 % at $n = 32$ and 56); $k = 32$ recovered only 60–70 %, which would have confused width with lost information. **checked**

> **! Caveat**
>
> The **inherited** and **budget** choices (feed-forward ratio, batch size, dropout and weight decay, number of seeds, the n range) were not varied; head size is fixed by design and is what Q5 would vary. The checked ones were, and none of them changed the main result.

3. Timeline

Date	Step	Outcome
09-24	Topic settled; literature sweep	35 papers, theory predictions (`docs/WIDTH-LITERATURE.md`)
09-30	Q1: fixed vs tuned LR (72 runs)	fixed LR distorts the width curve
10-01	Q1b: + warm-up/cosine (120 runs)	instability fixed; wide-model drop remains
10-01	Q1c: width × data (216 runs)	the drop is memorization; data moves the ceiling
10-01	Found the index leak in `hard`	Q1 measures a near-local task

10-02	Q3 pilot 1: `hard_diam` (18 runs)	diameter ≈ n/2; nothing learns at $n \geq 32$
10-02	Shortcut audit; `swap` generator	statistics at chance, ≤ 4-hop checks at 0
10-02	Q3 pilot 2: `swap`, 8000 graphs	17 of 18 runs stay trivial
10-02	Learnability probe (6 runs)	learns to 0.98–0.998 with 32 000 graphs
10-04	Q3 trimmed grid (60 runs)	clean floor at $n = 32$; onset width grows with n
10-05	Q3 fine grid ($n = 32$–56, 84 runs)	critical width $\propto n^{1.55}$ (fit), $n^{2.27}$ (generalize)
10-05	Q3 data check (32 runs)	128 000 graphs close the fit/ generalize gap
10-06	Q3 at 128k, $n = 32$ / 40 (24 runs)	width to learn $\propto n^{1.94}$ with enough data
10-07	Q3 step budget: 2× steps (16 runs)	critical width unchanged — not a training-time effect
10-07	Q3 learning rate: 10^{-3} (16 runs)	helps wide models at $n = 56$; slope $n^{1.94} \to n^{1.85}$
10-07	Q6: fixed 96-wide input (24 runs)	growth persists ($n^{2.0}$) — not the read-in
10-08	Q7a: depth 2–8 × width at $n = 40$ (60 runs)	depth 2 learns at $m \approx 1.1n$; depth saves ≈ ⅓ of the width
10-09	Q7c: depth 2 at $n = 32$ / 48 / 56 (34 runs)	depth-2 critical width $\propto n^{3.1}$ (fit), not linear
10-10	Sanford A: their GraphQA experiment (4 runs)	92.9 / 98.0 reproduced (93.0 / 98.2)
10-10	GraphQA shortcut audit	degree rule 0.973, 2-hop trees 0.995
10-11	Sanford B1: width 8–768 on GraphQA (12 runs)	98 at $m = 32$; hard cases flat in width
10-12	Sanford B2: their encoding on `swap` (56 runs)	no width ≤ 96 leaves always-"yes" in 24k steps
10-13	Sanford b2long: 100k steps, $m \leq 192$ (8 runs)	loss escapes into memorization; supervision confound found
10-13	Supervisor: study input encodings; Q7b dropped	encoding study (E) set up
10-14	`enc_edges`: edge tokens, all-pairs labels (40 runs)	no critical width at any n; ≈ 3× the width of adjacency rows
10-15	`enc_edges96k`: 4× the steps, $n = 32$ / 40 (16 runs)	learns: $m^* = 48$ / ≥ 72, 2.2× / ≥ 2.4× adjacency rows
10-15	Walk encodings: k-hop rows, RWSE (72 runs, Colab)	2 free hops: width ÷ 2.3–3.2; RWSE: no change; RWSE-16 leaks
10-20	More sizes: $n = 32$ / 48 (+ 72 for k-hop), LR 10^{-3} (95 runs)	growth $n^{1.74} \to n^{1.27} \to n^{1.03}$ with 0 / 2 / 4 free hops
10-21	Probing: 16 saved models, $n = 40$ / 56, linear probes per layer	connectivity builds up over all layers; no hop hierarchy; not stored per node

Table 1: What was done, in order.

4. Q1 — the learning rate confounds width

Set-up. $n = 24$, depth 2, 2000 training graphs, widths 8–256, learning rates $3 \cdot 10^{-4}$...10^{-2}, 3 seeds (Q1). Q1b repeats it with 5 % linear warm-up and cosine decay, adds widths 2 and 4 and rate $3 \cdot 10^{-2}$.

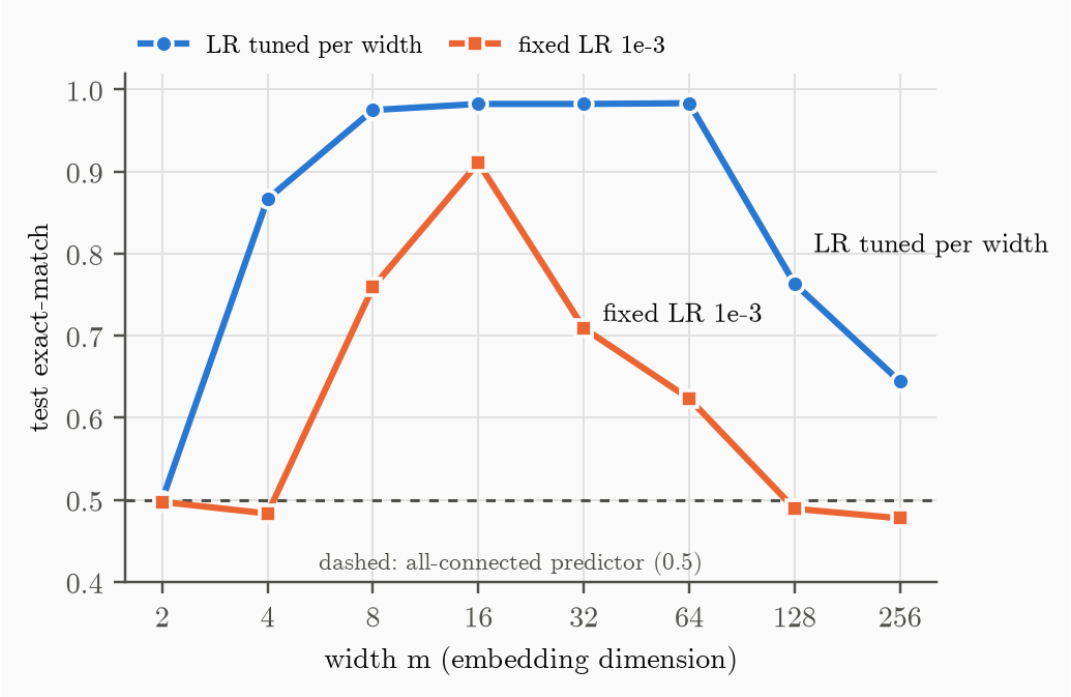


Figure 1: Q1b: test exact-match vs width with one fixed learning rate versus the rate chosen per width on validation (mean of 3 seeds).

m	2	4	8	16	32	64	128	256
Q1, fixed 1e-3	–	–	0.917	0.917	0.781	0.912	0.820	0.596
Q1, tuned	–	–	0.976	0.977	0.967	0.948	0.844	0.678
Q1b, fixed 1e-3	0.497	0.483	0.759	0.911	0.709	0.623	0.489	0.478
Q1b, tuned	0.497	0.867	0.975	0.967	0.982	0.983	0.763	0.644

Table 2: Test exact-match, mean of 3 seeds.

✓ **Finding**

- With a fixed rate the width curve zig-zags and collapses (Q1b: 0.48 at $m \geq 128$); tuned per width, $m =$ 8–64 all reach $\approx$ 0.97–0.98.
- The best rate **falls as width grows** (10^{-2} at small $m \to 10^{-3}$ at $m = 256$); without warm-up, wide models diverge at high rates ($m = 256$ at 10^{-2}: train 0.41).
- Warm-up fixes the instability (wide models reach train 1.00) — but tuned test at $m = 128/256$ stays at 0.76 / 0.64.

5. Q1c — the wide-model drop is memorization

Same set-up with 500, 2000 and 8000 training graphs, at a **fixed number of optimizer steps** (4800), so more data never means more training.

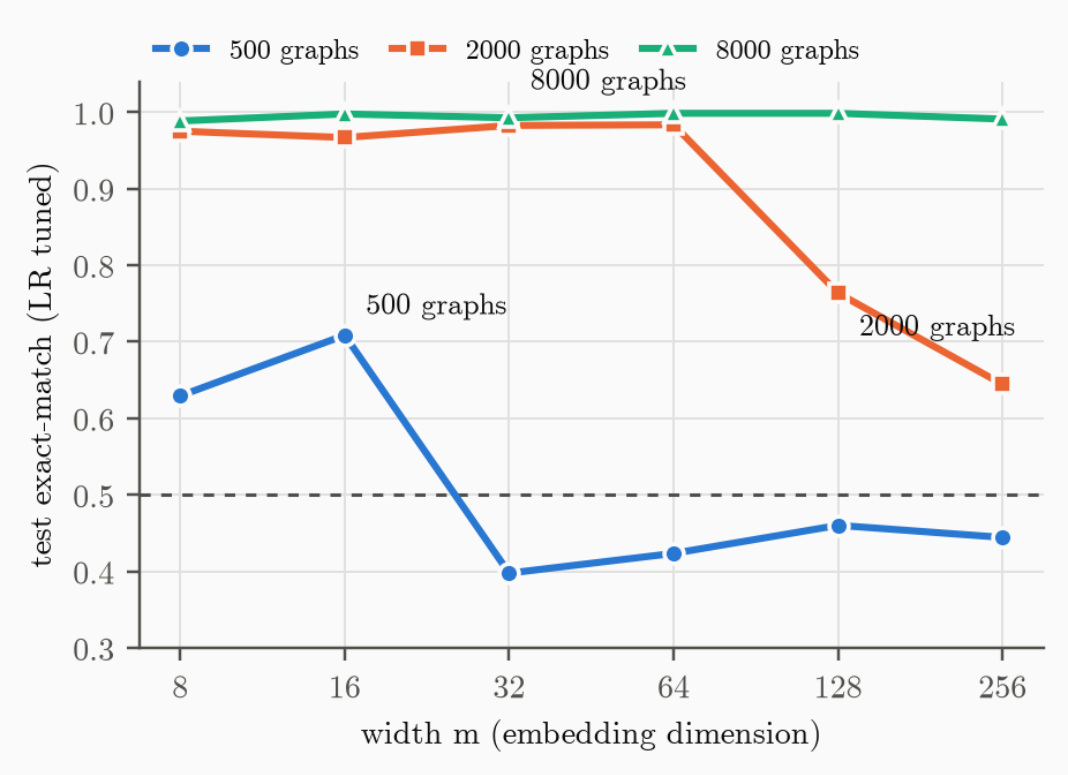


Figure 2: Q1c: tuned test exact-match vs width for three training-set sizes. Training accuracy is ≈ 1.00 everywhere.

✓ Finding

- At 8000 graphs every $m \geq 16$ reaches ≥ 0.99 — the drop at $m \geq 128$ is gone.
- At 500 graphs everything memorizes; $m \geq 32$ falls **below** the trivial predictor.
- The 2000-graph arm reproduces Q1b to the third decimal (independent Kaggle run).
- With enough data the fixed rate is within 0.01 of tuned; tuning matters most when data is scarce.

◆ What it means

Width has a **floor** (below ≈ 4–8 the model cannot even fit the training set) and a **ceiling set by the data** (past it, extra width memorizes). The floor is what theory talks about; the ceiling is the practical limit it doesn't cover.

! Caveat

Q1 used the `hard` generator, which turned out to leak (next section). Q1′s lessons are about **method** — tune the rate per width, give enough data — not about reachability.

6. Making the task honest: the shortcut audit

The `hard` generator builds two dense blobs, with or without one bridge. Three problems surfaced:

1. **Index leak.** Node labels were never shuffled: the blobs are always nodes $[0, n_a)$ and $[n_a, n)$ (200 / 200 graphs checked). With adjacency-row tokens the split point is readable, so connectivity reduces to "does any row have ones on both sides".
2. **Statistics leak.** A bridge closes no cycle; the matched extra edge in the disconnected class always closes a short one. Gradient boosting on degree and closed-walk counts ($\operatorname{tr} A^3 \ldots \operatorname{tr} A^6$) predicts the label with 0.68–0.90 accuracy at $n = 16$ for every existing generator.
3. **Locality.** On small graphs a model that only looks 6 hops away gets the whole matrix right on 60–100 % of graphs.

Fix — the `swap` generator. Two sparse blobs (cycle + ¼ · size chords). Pick two nodes ≥ 6 hops apart in each blob (a_1, a_2 and b_1, b_2). Disconnected: add a_1a_2 and b_1b_2. Connected: add a_1b_1 and a_2b_2. Same degree changes in both classes; every new cycle has ≥ 7 edges; graphs failing the distance test are redrawn **before** the label is applied; node labels shuffled per graph. The idea — classes that no local statistic can tell apart, two cycles vs one — is the "cycle task" Abbe et al. (NeurIPS 2024) proposed as a training benchmark; `swap` adds chords (so diameter grows only $\approx \log n$), a dense pair target, and an explicit audit.

n	32	40	48	56	64	128
statistics → label	0.526	0.512	0.493	0.504	0.512	0.507
statistics + 4-hop (exact)	0.256	0.257	0.246	0.256	0.259	0.253
4-hop check (exact)	0.000	0.000	0.000	0.000	0.000	0.000
diameter, connected	8.6	9.8	10.9	11.7	12.5	16.7

Table 3: Audit of `swap` (chord fraction 0.25, $d_{\min} = 6$) — `audit_width_data.py`. Trivial predictor: 0.5. Diameter grows $\approx \log n$.

◆ **What it means**

Every heuristic we could think of sits at or below the trivial predictor, so a model that scores well on `swap` has to trace paths. The price: `swap` cannot be built below $n \approx 32$, and it is much harder to learn.

7. Q3 — critical width versus graph size

Pilots and the learnability probe

Run	Set-up	Result
Pilot 1	`hard_diam` (diameter ≈ n/2), n = 16/32/64, 8000 graphs	$n = 16$ learns (depth 4: 0.99); $n \geq 32$ all at 0.50
Pilot 2	`swap`, n = 32/64/128, 8000 graphs, 4800 steps	17 / 18 at 0.50; only $n = 32$, depth 5, $m = 128$: pair 0.87 vs 0.75 trivial
Probe	`swap`, $n = 32$, depth 5, 32 000 graphs, 24 000 steps	m = 64 / 128 / 256 reach 0.985 / **0.998** / 0.988 test exact-match

Table 4: The road to a learnable, shortcut-free setting.

✓ **Finding**

The clean task **is** learnable — test tracks train (0.998 vs 1.000), pair accuracy 0.998. Pilot 2 failed mainly for lack of data (with 8000 graphs the $m = 128$ model memorized: train 0.99, test 0.53), the same data ceiling as Q1c. Wider models also start later at a fixed rate (rate $3 \cdot 10^{-3}$, epoch 12: 0.83 / 0.40 / 0.07 for m = 64 / 128 / 256).

The trimmed grid

`swap`, n = 32 / 64 / 128 at depth $\lceil \log_2 n \rceil$ = 5 / 6 / 7, m = 8 … 128, 32 000 graphs, 12 000 steps, rates 10^{-3} and $3 \cdot 10^{-3}$, 2 seeds (60 runs).

m	$n = 32$	$n = 64$	$n = 128$
8	0.50 (train 0.27)	0.50	0.50
16	0.49 (train 0.39)	0.50	0.50
32	**0.90**	0.50	0.50
64	**0.97**	0.50	0.50
128	**0.97**	0.50	0.50

Table 5: Tuned test exact-match (mean of 2 seeds).

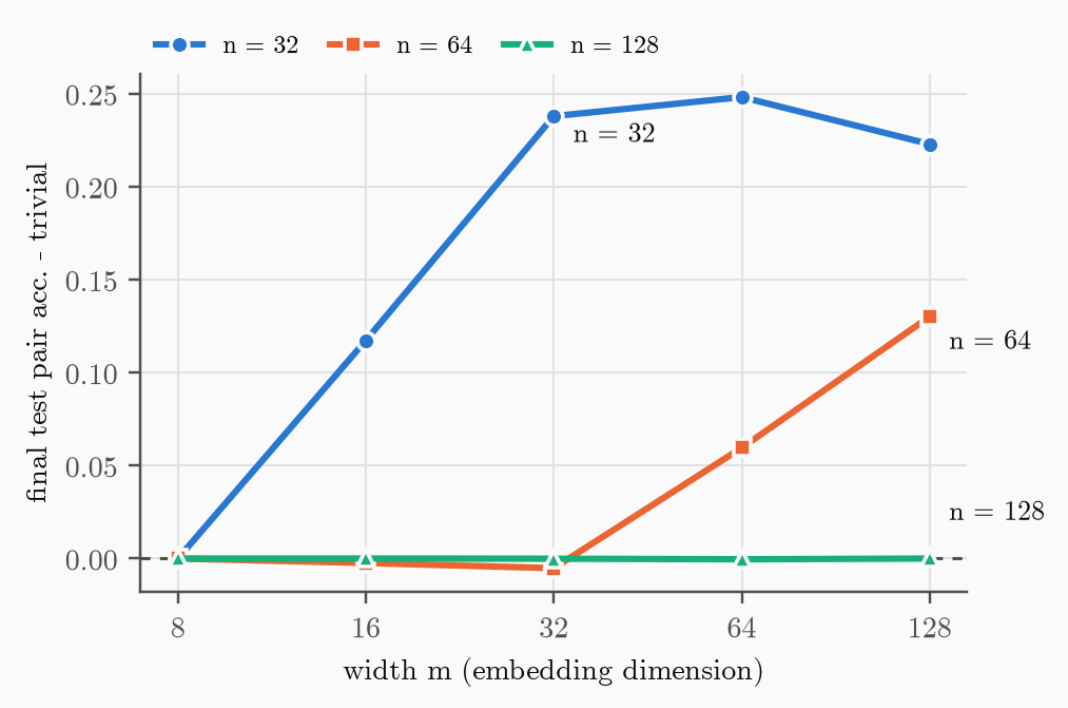


Figure 3: How far each model gets above the trivial predictor (final test pair accuracy at rate $3 \cdot 10^{-3}$, mean of 2 seeds, no selection on test). The width at which learning starts moves right as n grows.

✓ **Finding**

- $n = 32$: a sharp floor. $m = 8$ cannot fit the training set; $m = 16$ starts but stalls; $m = 32$ reaches 0.90; $m \geq 64$ reach 0.97. $\boldsymbol{m^*}$ **is between 32 and 64** (by pair accuracy: 32 at ≥ 0.95, 64 at ≥ 0.99).
- $n = 64$: $m \leq 32$ never leave the trivial predictor; $m = 64$ starts (train loss 0.59 → 0.07–0.10, pair 0.80–0.85); $m = 128$ gets further (pair 0.89–0.90). Exact-match stays 0.5 — about 4000 pairs must all be right.
- $n = 128$: nothing moves for any $m \leq 128$.

◆ **What it means**

The width at which learning **begins** is ≈ 16 at $n = 32$, ≈ 64 at $n = 64$ and above 128 at $n = 128$ — growing at least linearly in n. This is the direction of the theory: a single step that gathers a node's whole reach set is q-sparse averaging with q close to n, which needs width of order n (Sanford, Hsu, Telgarsky 2023; see `reports/qsa-sparse-averaging.pdf`). On the old, leaky task the same model needed only $m = 8$ at $n = 24$ — the shortcuts were hiding the width requirement.

The fine grid: a scaling law

`swap`, $n = 32$ / 40 / 48 / 56 at a fixed depth 6 ($= \lceil \log_2 n \rceil$ for all four), $m = 16$ … 128, 32 000 graphs, 24 000 steps, rate $3 \cdot 10^{-3}$, 3 seeds (84 runs).

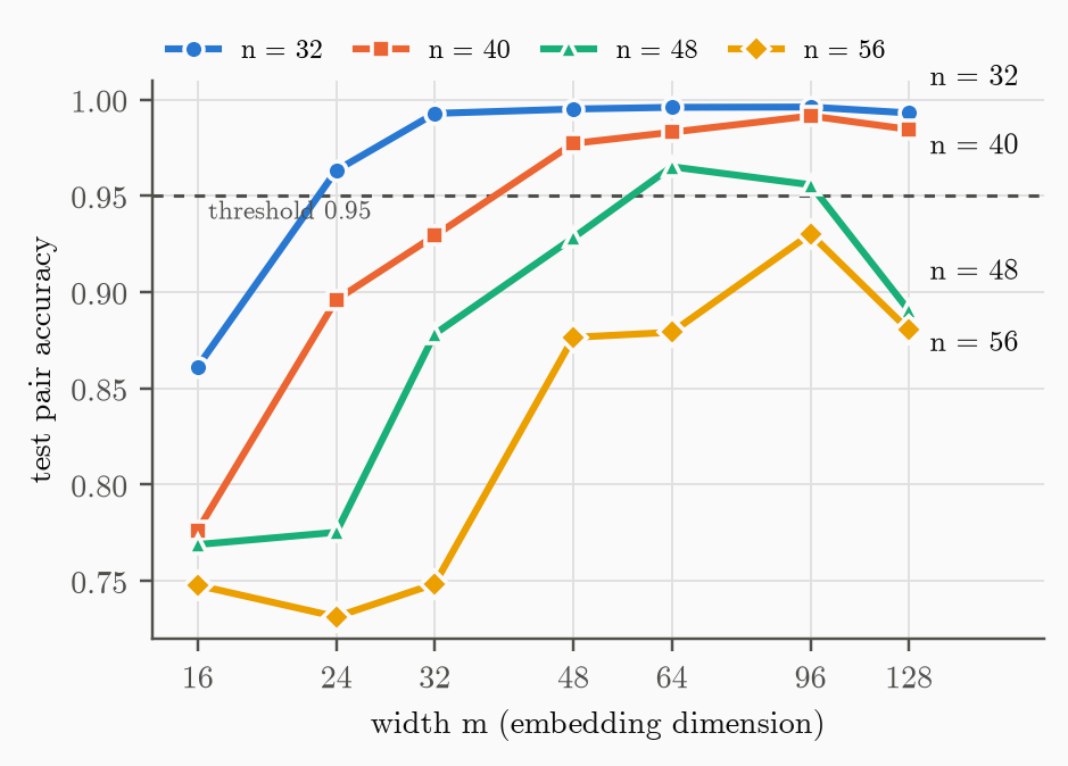


Figure 4: Final-epoch test pair accuracy vs width (mean of 3 seeds; trivial ≈ 0.75). Each curve shifts right as n grows. The drop at $m = 128$ is the learning-rate effect from Q1: $3 \cdot 10^{-3}$ is too high there, and training accuracy falls too.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
width to **fit** (final train pair ≥ 0.95)	18.6	23.5	31.7	44.7	$\propto n^{1.55}$
width to **generalize** (final test pair ≥ 0.95)	22.9	37.6	57.5	> 96	$\propto n^{2.27}$

Table 6: Critical width: mean over seeds of the width where each seed's curve crosses 0.95 (interpolated in $\log_2 m$); slope of the log–log fit across n ($m \leq 96$; `width_sweep.py q3fine --scaling`). Per-seed crossings agree within ≈ ±3.

! Caveat

Correction (2026-10-05). An earlier version of this report read test pair accuracy at the checkpoint with the best validation **exact-match**. When exact-match never rises above 0.5, every epoch ties and epoch 1 is chosen, so the trained model's pair accuracy was replaced by the trivial predictor's (e.g. $n = 48$, $m = 32$: reported 0.755, actually 0.885 / 0.853). All pair-accuracy numbers now use the final epoch. The critical widths barely move (generalize slope 2.19 → 2.27); the curves figure changed most. New runs also log validation pair accuracy.

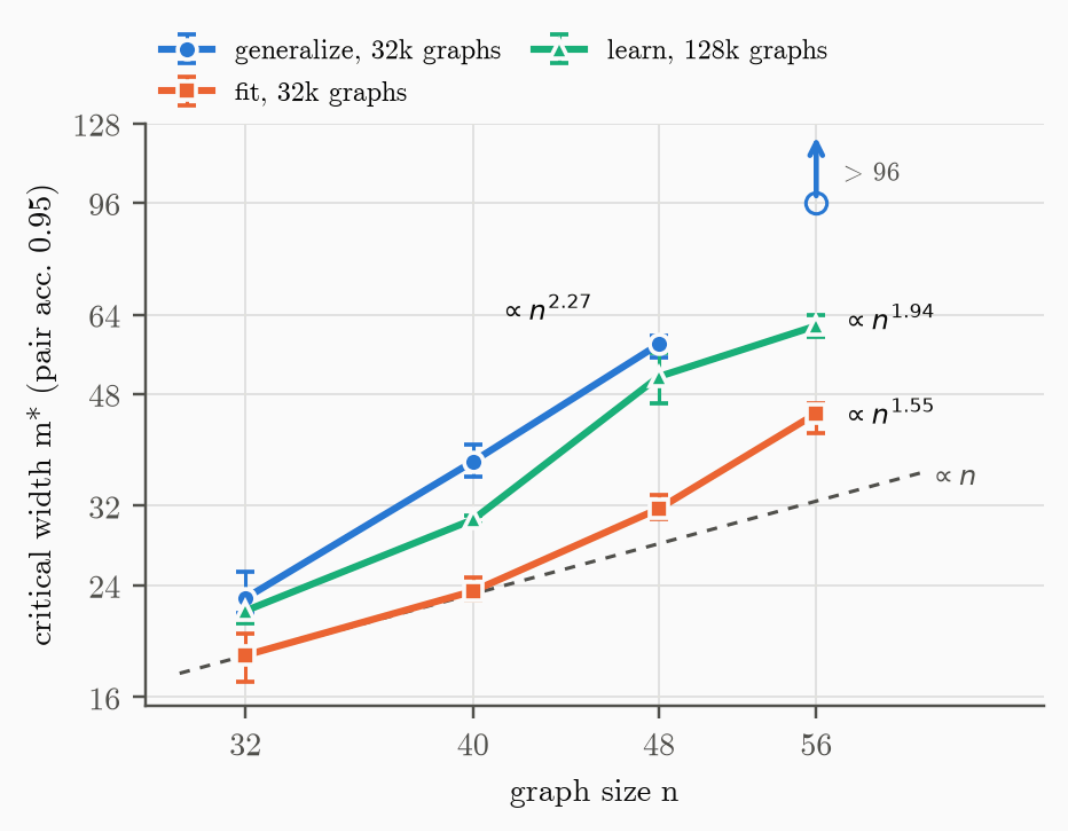


Figure 5: Critical width vs graph size, log–log (error bars: range over seeds). With 32 000 graphs, fitting and generalizing come apart; with 128 000 graphs (green; data check and the $n = 32$ / 40 runs below) they coincide, and the width to learn still grows as $n^{1.94}$ — steeper than $m \propto n$ (dashed).

✓ Finding

- Both critical widths grow **faster than linearly** in n: $\approx n^{1.5}$ to fit, $\approx n^{2.3}$ to generalize.
- The gap between them widens with n: at $n = 56$, $m = 64$ fits the training set (pair 0.984) but reaches only 0.879 on test.
- Seeds agree closely; the scaling is not noise.

◆ What it means

Theory says depth $\log N$ needs width only just above $\sqrt{N}$ to **represent** connectivity, with no extra tokens (N = vertices + edges in their encoding; Sanford et al. 2024a, Thm 18 — width N^{ε} for small ε needs $\approx N^{4-\varepsilon}$ extra "pause" tokens, which our models do not have). What we measure is the width to **learn** it, and that grows faster than n. The widening fit/generalize gap is the Q1c data ceiling again: 32 000 graphs give fewer examples per pair as n grows, so the 2.27 is probably inflated by holding data fixed. The data check tests this.

The data check: more data closes the gap

Same set-up at $n = 48$ / 56, $m = 32$ … 96, with 64 000 and 128 000 graphs at the same 24 000 steps (2 seeds). The larger training sets start with the same graphs as the smaller ones; validation and test sets are identical.

	critical width	32k graphs	64k graphs	128k graphs
$n = 48$	to fit	31.7	40.5	48.4
	to generalize	57.5	54.1	**51.1**
$n = 56$	to fit	44.7	57.8	59.4
	to generalize	> 96	> 96 †	**61.5**

Table 7: Critical width (mean of per-seed crossings at 0.95, final epoch). 32k: 3 seeds; 64k / 128k: 2 seeds. † one seed 81.8, the other above 96.

✓ **Finding**

- **The gap closes.** At 128 000 graphs the widths to fit and to generalize nearly meet: 48 vs 51 at $n = 48$, 59 vs 61 at $n = 56$. At $n = 56$, $m = 64$: train / test pair 0.984 / 0.879 with 32k graphs → 0.975 / 0.927 with 64k → 0.972 / 0.960 with 128k.
- **Fitting gets harder with more data at fixed steps** (each graph is seen fewer times), so the fit width rises: 31.7 → 48.4 at $n = 48$.
- At 128k the width to **learn** is ≈ 51 and ≈ 61 for $n = 48$ and 56, about $1.1n$.

◆ **What it means**

The widening gap at 32 000 graphs was the data ceiling: with enough data, fitting and generalizing measure the same thing — the width to **learn** connectivity. Two sizes 17 % apart cannot fix a slope, so the same 128 000-graph runs were repeated at $n = 32$ / 40.

Four sizes with enough data

$n = 32$ / 40 at 128 000 graphs, $m = 16$ … 96, same 24 000 steps and rate, 2 seeds (24 runs), joined with the data check's $n = 48$ / 56.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
width to fit	20.9	29.5	48.4	59.4	$\propto n^{1.95}$
width to generalize	21.9	30.4	51.1	61.5	$\propto n^{1.94}$
generalize m^*/n	0.68	0.76	1.07	1.10	

Table 8: Critical widths at 128 000 graphs (mean of per-seed crossings at 0.95, final epoch). Resampling one seed per n gives a 90 % range of 1.74–2.14 for the generalize slope.

✓ **Finding**

- With enough data, fitting and generalizing **coincide at every n** (within ≈ 2).
- The shared critical width grows as $n^{1.94}$: from $0.68n$ at $n = 32$ to $1.10n$ at $n = 56$. The growth is uneven — steepest between $n = 40$ and 48 — and $n = 48$ has the largest seed spread (46.5 vs 55.8).

◆ **What it means**

The superlinear growth is **not** a data artefact: removing the data ceiling closed the gap but left the slope near 2. Over this range, the width a transformer needs to **learn** connectivity grows roughly quadratically in n, while the width needed to **represent** it at depth $\log N$ is sublinear — just above $\sqrt{N}$ without extra tokens (Sanford et al. 2024a). The remaining confounds are the fixed step budget (24 000 steps; more training might let narrower models get there — tested next) and the single learning rate.

The step budget: more training does not lower the critical width

Same setting at 128 000 graphs with **48 000** steps instead of 24 000, $n = 40$ / 56, on the widths around each threshold (2 seeds, 16 runs).

	critical width	24 000 steps	48 000 steps	change
$n = 40$	to fit	29.5	26.9	−2.6
	to generalize	30.4	**29.8**	−0.6
$n = 56$	to fit	59.4	56.3	−3.1
	to generalize	61.5	**67.4** †	+5.9

Table 9: Critical width (mean of per-seed crossings at 0.95, final epoch). † seeds 62.7 / 72.0 — within the seed spread.

✓ **Finding**

- Twice the training lowers the width to **fit** by only ≈ 3 and leaves the width to **generalize** unchanged within the seed spread.
- At $n = 56$ the longer run starts to memorize again (48 passes over the data): $m = 64$ goes from train / test pair 0.972 / 0.960 to 0.975 / 0.948.
- The slope between $n = 40$ and 56 at 48 000 steps (≈ 2.2 fit, ≈ 2.4 generalize) is consistent with the four-point $n^{1.94}$.

◆ **What it means**

The width requirement is not narrower models simply learning more slowly.

The learning rate: tuning trims the slope, not the trend

Same setting at 128 000 graphs and 24 000 steps with rate 10^{-3} instead of $3 \cdot 10^{-3}$, $n = 40$ / 56, same widths (2 seeds, 16 runs). "Tuned" picks the rate per width and seed on final validation exact-match, as in Q1.

	critical width	$3 \cdot 10^{-3}$	10^{-3}	tuned
$n = 40$	to fit	29.5	28.2	28.0
	to generalize	30.4	30.1	**30.2**
$n = 56$	to fit	59.4	52.7	52.7
	to generalize	61.5	57.9	**57.9**

Table 10: Critical width (mean of per-seed crossings at 0.95, final epoch).

✓ **Finding**

- At $n = 40$ the rate does not matter: the curves coincide.
- At $n = 56$ the higher rate was holding the wide models back, as Q1 predicted: $m = 96$ goes from test pair 0.932 to **0.984**. Validation picks 10^{-3} for every $m \geq 48$ at $n = 56$.
- Slope between $n = 40$ and 56: 2.09 → 1.93. Four-point slope with the tuned values at $n = 40$ / 56 (and $3 \cdot 10^{-3}$ at 32 / 48, the only rate run there): $\boldsymbol{n^{1.85}}$, seed-bootstrap 90 % range 1.68–2.03.

◆ **What it means**

The superlinear growth survives every confound we have tested: the learning rate (Q1 and this check), the amount of data (data check) and the training budget (step check). **Update 10-20:** with $n = 32$ / 48 re-run at 10^{-3} as well, the critical width is 21.9 / 30.1 / 42.3 / 57.9 at $n = 32$ / 40 / 48 / 56 — $n = 48$ came down from 51.1, $n = 32$ did not move — and grows as $\boldsymbol{n^{1.74}}$ (90 % 1.59–1.90) at one rate throughout.

8. Q6 — is the growth just the input encoding?

Each node token is its adjacency row, n numbers wide, and the model's first layer projects it to width m. The critical width we measured is ≈ 0.7–1.1n — close to where that projection stops compressing — so the growth might come from the encoding, not the task.

Test. Feed each node $(A + I)P$ instead, with P a fixed random $n \times 96$ matrix ($\pm 1/\sqrt{96}$ entries, one "ID" vector per node, drawn per seed, never trained). The input is 96 wide for every n and still determines each neighbour set (simple decoding recovers ≈ 99 % of rows at $n = 32$ and 56; a 32-wide code recovers only 60–70 %, which would have confounded width with lost information). Everything else is the tuned 128 000-graph baseline: same graphs, model, 24 000 steps, rate 10^{-3}, $n = 40$ / 56, $m = 16$ … 96, 2 seeds (24 runs).

critical width	$n = 40$	$n = 56$	growth 40 → 56
baseline: adjacency rows (n wide), tuned	30.2	57.9	$\propto n^{1.93}$

Q6: projected IDs (96 wide), to generalize	37.3 (32.3 / 42.2)	73.8 (69.8 / 77.8)	$\propto n^{2.03}$
Q6: projected IDs, to fit	37.0	57.8	$\propto n^{1.33}$

Table 11: Critical width (mean of per-seed crossings at 0.95, final epoch; per-seed values in brackets).

✓ **Finding**

- With an input whose size does not depend on n, the width to generalize still roughly doubles from $n = 40$ to 56 ($n^{2.03}$, vs $n^{1.93}$ for the baseline).
- The projected input needs ≈ 20–25 % more width at both sizes: a random code of the neighbour set has to be decoded before it is usable. It shifts the curve up without changing its slope.
- Seed spread is larger than for the baseline (32 vs 42 at $n = 40$), so the two-point slope is rough; the doubling itself is clear.

◆ **What it means**

The growth is not an artefact of the n-wide read-in: it belongs to the task (or to the architecture beyond the first layer). Across two input encodings, learning connectivity needs width growing roughly as n^2 over this range.

9. Q7a — can depth replace width?

$n = 40$, the audited `swap` data (connected diameter ≈ 10), 128 000 graphs, 24 000 steps, rate 10^{-3}, depth $L \in \{2, 3, 4, 6, 8\}$, widths around each threshold, 2 seeds (60 runs).

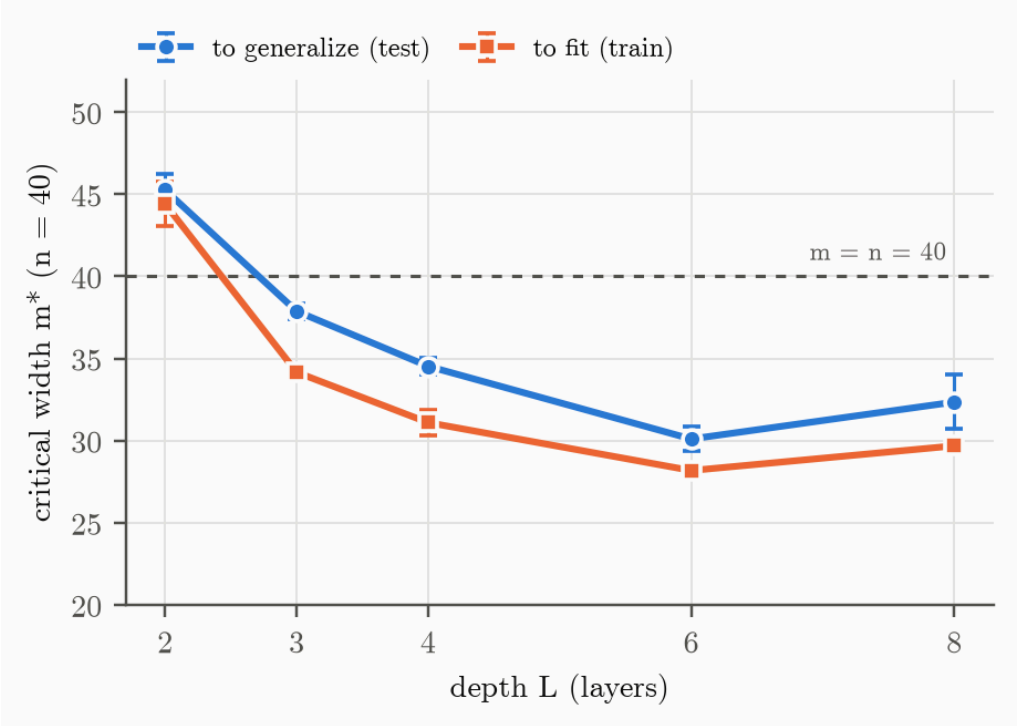


Figure 6: Critical width vs depth at $n = 40$ (error bars: range over two seeds).

depth L	2	3	4	6	8
critical width to generalize	45.3	37.9	34.5	30.1	32.3
m^*/n	1.13	0.95	0.86	0.75	0.81

Table 12: Mean of per-seed crossings at 0.95, final epoch; seeds agree within ≈ ±2.

✓ **Finding**

- **Two layers are enough.** At $L = 2$, $m = 48$ reaches test pair 0.968 and $m \geq 64$ reach ≈ 0.99 — on graphs of diameter ≈ 10. The prediction "depth must cover the diameter by doubling, $2^L \geq 10$" was wrong. Ye et al. (2026) prove a tighter reach: an L-layer model can handle diameter up to 3^L, so two layers already cover ≈ 9 — right at our graphs' diameter, which fits depth helping little beyond 3. (Their bound is for a restricted architecture; our standard 2-layer models still reach 0.99.)
- **At this size the trade-off is mild and saturates.** Going from 2 to 6 layers saves about a third of the width (45 → 30); depth 8 is no better than 6. (Q7c: the saving grows with n.)
- **Exact reproduction.** The $L = 6$ runs repeat the Q3 learning-rate baseline at $n = 40$ and give the same critical width (30.1).

◆ **What it means**

With adjacency-row tokens, width $\approx n$ makes constant depth sufficient — the regime Yehudai et al. (2025) prove for their tokenization ("linear width, constant depth"); we measure the constant: $\approx 1.1n$ at depth 2. At fixed n depth matters little, yet at depth 6 the critical width grows $\approx n^2$ across sizes. Q7c asks how the depth-2 critical width grows with n.

10. Q7c — does the depth-2 width grow linearly?

Same setting as Q7a (the audited `swap` data, 128 000 graphs, 24 000 steps, rate 10^{-3}) at depth 2 for $n = 32$ / 48 / 56, widths 16–192 placed to cover both predictions ($m^* \propto n$ and $m^* \propto n^2$), 2 seeds (34 runs); Q7a's depth-2 runs supply $n = 40$.

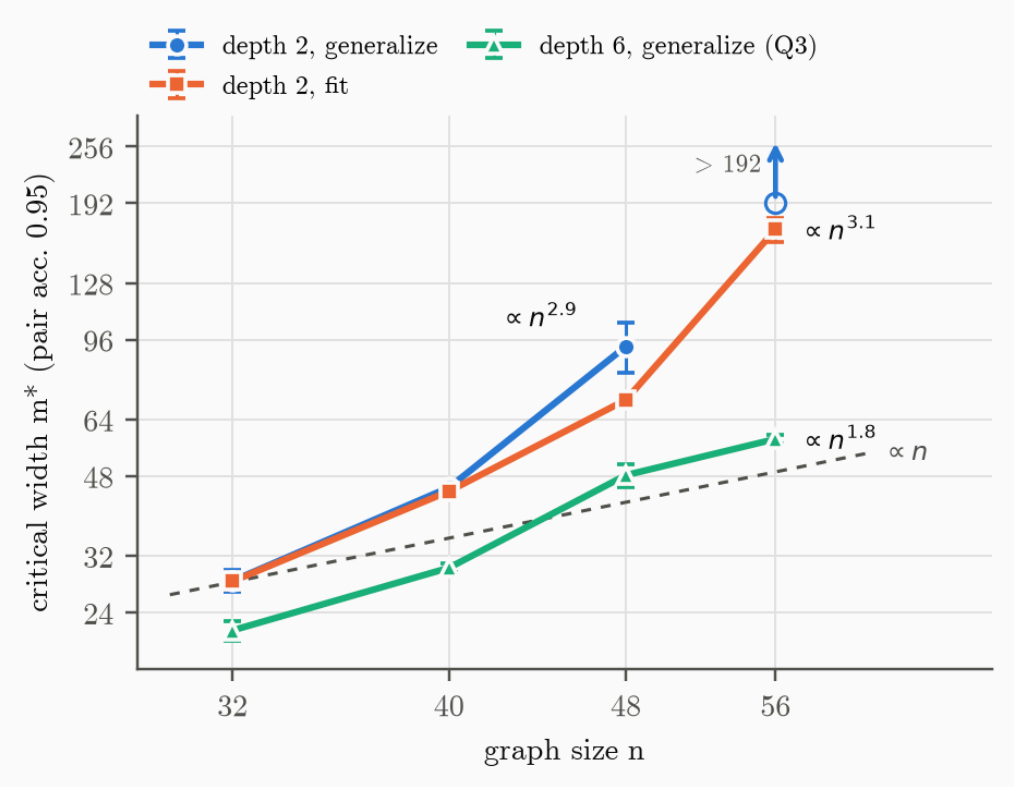


Figure 7: Critical width vs n at depth 2 and depth 6 (log–log; error bars: range over two seeds; open circle: no seed reaches 0.95 by $m = 192$). Depth 6 is Q3 at 128 000 graphs, rate 10^{-3} at $n = 40$ / 56 and $3 \cdot 10^{-3}$ at 32 / 48.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth

depth 2, to fit	28.1	44.4	70.7	168.0	$\propto n^{3.07}$
depth 2, to generalize	28.2	45.3	92.7	> 192	$\propto n^{2.91}$ (to 48)
depth 6, to generalize	21.9	30.2	51.1	57.9	$\propto n^{1.85}$
depth 2 ÷ depth 6	1.3	1.5	1.8	> 3.3	

Table 13: Critical widths (mean of per-seed crossings at 0.95, final epoch). Taking one seed per n gives fit slopes of 2.90–3.24.

✓ **Finding**

- **Not linear.** The depth-2 critical width grows as $\approx n^3$ — faster than depth 6's $\approx n^{1.9}$. From $1.1n$ at $n = 40$ it reaches $3n$ at $n = 56$.
- **The robustness is in the direction, not the exponent.** With a looser threshold the fit slope is 2.4 (pair accuracy 0.90) or 2.8 (0.93): always above depth 6's and far above linear.
- **The step to $n = 56$ is the steepest.** Local fit slopes are 2.1 ($32 \to 40$), 2.6 ($40 \to 48$) and 5.6 ($48 \to 56$). At $n = 56$ the curve is flat near the threshold — width 192 only reaches train pair 0.96 — so this point is the least certain (seeds: 158 and 178).
- **Fitting and generalizing separate again at depth 2, even with 128 000 graphs.** At $n = 56$, $m = 192$ has train pair 0.96 but test pair 0.92; at depth 6 the same data closed this gap.

◆ **What it means**

The constant-depth, linear-width solution exists for this tokenization (Yehudai et al. 2025), but training does not find it as n grows: two layers need ever more width than six. So Q7a's "depth trades only mildly" holds only at $n = 40$; the width that depth saves grows with n, from $\approx 1.3\times$ at $n = 32$ to more than $3\times$ at $n = 56$. This fits the log-depth constructions (Sanford et al. 2024a): with too few layers, width has to stand in for depth at a rising cost.

11. Reproducing Sanford et al. 2024a

The supervisor asked whether the experiment behind Sanford et al.'s claim — connectivity at depth $O(\log N)$ with width $\approx \sqrt{N}$ — had been reproduced. Their model is fixed (decoder, $L = 12$, $m = 768$, GLU, vertex + edge + task tokens) and trained on GraphQA connectivity (ER graphs, $n \leq 19$, one s–t question) at 1K and 100K graphs. No code was released; we re-implemented data, tokens and model (`sanford_repro.py`, details in `docs/sanford-reproduction.md`).

	Theirs	**Ours**
test accuracy, 1K graphs	92.9	**93.0**
test accuracy, 100K graphs	98.0	**98.2** (learning rate 10^{-4}, not their $5 \cdot 10^{-4}$)
their rate at 100K	—	stuck at 0.93, then NaN
rule: both endpoints have an edge	—	0.973
boosted trees on 2-hop neighbourhoods	—	**0.995**
width needed for 98 (B1, $m = 8 \ldots 768$)	never varied	$m = 32$

Table 14: Part A reproduces both numbers; the audit and B1 show what they measure.

✓ **Finding**

- **Both numbers reproduce** (within ± 0.3 pt), the 100K one only at a lower learning rate.
- **GraphQA connectivity is local.** A degree rule gets 0.973 and a 2-hop classifier 0.995 — above the reproduced 71M-parameter transformer. 94 % of connected pairs are ≤ 2 hops apart.
- **Width only buys the shortcut** (B1). Test accuracy reaches 0.98 at $m = 32$ and stays flat to 768; on a hard set (pairs ≥ 4 hops apart, no-path pairs with edges) every width sits at 0.81–0.89 / 0.63–0.70, below the 2-hop classifier.

B2 — their encoding on audited data. Q3′s setting (`swap`, $n = 32$–56, depth 6, 128 000 graphs, 24 000 steps, rate 10^{-3}) with their tokens instead of adjacency rows: n vertex + $|E|$ edge tokens and 32 query tokens per graph, each read out at its own position. Local classifiers sit at the always-"yes" rate (≈ 0.75); 0.95 needs $\approx$ 8 hops of reach. $m = 8$ … 96, 2 seeds (56 runs).

final training loss (seed 0)	$m = 8$	$m = 32$	$m = 64$	$m = 96$
$n = 32$	0.567	0.567	0.556	**0.532**
$n = 40$	0.562	0.561	0.559	0.548
$n = 48$	0.557	0.557	0.552	0.539
$n = 56$	0.552	0.551	0.546	0.537

Table 15: B2: every run ends at the always-"yes" accuracy (test 0.74–0.76); the always-"yes" loss is ≈ 0.568 at $n = 32$ and 0.552 at $n = 56$.

✓ **Finding**

- **No width up to 96 learns**, at any n — the same budget at which adjacency rows cross 0.95 at $m =$ 22–58 (Q3). Labels and read-out were checked against the tokens.
- **The escape has begun, in width order.** The loss leaves the trivial level late and further for wider models; at $n = 32$, $m = 96$ is flat to step 14 000 and then falls.

Longer budget (b2long). 100 000 steps, $n = 32$ / 48, $m = 32$ / 96 / 192, rates 10^{-3} and $3 \cdot 10^{-4}$ (8 runs). The loss now leaves the trivial level (after 20 000–65 000 steps, earlier for wider models and the lower rate), but only into memorization: training loss falls to 0.27–0.48 while training accuracy moves little ($\lesssim 0.86$) and test accuracy drops **below** always-"yes" (0.64–0.68 at $m \geq 96$). No run generalizes.

! **Caveat**

Supervision confound. Q3 trains on the whole reachability matrix — $n^2 = 1024$–3136 labels per graph — while B2 gives 32 query labels per graph, 32–98× fewer on the same 128 000 graphs. B2′s failure therefore mixes "their encoding is harder" with "far less signal per graph"; the memorization fits a model starved of labels.

◆ **What it means**

Their experiment reproduces, but it does not test their claim: GraphQA is solved locally, and $m = 32$ suffices for $n \leq 19$. On data that requires reachability, their set-up (task tokens, 32 questions per graph) does not learn at any width up to 192 — far from the representation bound ($m \approx \sqrt{N}$, $\approx$ 9–11). Whether the **encoding** is to blame needs the same supervision as Q3: their tokens with an all-pairs read-out (next section).

12. Encodings — Sanford's edge tokens with all-pairs labels

The thesis now compares input encodings (supervisor, 10-13). The first is Sanford et al.'s: one token per vertex (a learned ID) and one per edge (the sum of its two vertex IDs), no positions, with the same depth-6 encoder and all-pairs read-out as Q3 — B2 without its 32-label confound. Protocol as q3lr: 128 000 `swap` graphs, 24 000 steps, rate 10^{-3}, $n = 32$–56, $m = 8$–96, 2 seeds (40 runs). Edge tokens make the sequence $\approx 2.25n$ long, $\approx 3\times$ the step cost.

n	$m = 16$	32	48	64	96	adjacency rows, same budget
32	0.745	0.820	0.921	—	—	critical width 21.9
40	0.750	0.758	0.790	0.851	—	$m = 24$: 0.919, $m = 48$: 0.985
48	0.755	0.754	0.757	0.782	0.804	critical width 51.1
56	0.759	0.759	0.759	0.759	0.775	$m = 32$: 0.821, $m = 96$: 0.984

Table 16: Edge tokens: final test pair accuracy, mean of 2 seeds (always "not connected" ≈ 0.75).

✓ **Finding**

- **No critical width at any n.** The only run past the trivial exact-match is $n = 32$, $m = 48$, one seed (0.76, pair 0.968, still rising); the other seed is at 0.10.
- **A long plateau, then slow progress in width order.** The loss sits at ≈ 0.55 for 7 000–18 000 steps (adjacency rows: 3 000–8 000) and is still falling at 24 000; train ≈ test, so the models are under-trained, not memorizing.
- **≈ 3× the width for the same accuracy**: at $n = 40$, edge tokens at $m = 64$ (0.851) sit between adjacency rows at $m = 16$ and 24; at $n = 56$, $m = 96$ (0.775) is below adjacency rows at $m = 32$.

◆ **What it means**

With the supervision equalized, Sanford et al.'s encoding still fails where adjacency rows succeed, so B2's failure was the encoding, not only the label count. An adjacency row hands each token its whole neighbourhood; with edge tokens the model must first learn to match IDs to gather it (a plateau we also see on the 1-hop task), and that step alone costs width and time. Their $\sqrt{N}$ bound is for this encoding — the one that is hardest to learn here.

Four times the steps (`enc_edges96k`). The models were still improving at 24 000 steps, so we gave $n = 32$ / 40 96 000 steps ($m = 32$–96, 2 seeds; a 7 h cap stopped the $m = 96$ runs at 48 000–54 000 steps and some others at 84 000–90 000).

m	$n = 32$	$n = 40$
32	0.52 / 0.939 · 0.50 / 0.915	0.37 / 0.856 · 0.22 / 0.762
48	0.74 / 0.972 · 0.36 / 0.816	0.14 / 0.757 · 0.41 / 0.856
64	**0.995 / 0.999 · 0.897 / 0.990**	0.59 / 0.934 · 0.11 / 0.788
96	0.835 / 0.979 · 0.960 / 0.996	0.87 / 0.985 · 0.47 / 0.906
critical width	**48.2** (adjacency rows: 21.9)	≥ **72** (adjacency rows: 30.2)

Table 17: Edge tokens at 96 000 steps: final test exact / pair accuracy, seed 0 · seed 1. Critical width as for Q3 (test pair ≥ 0.95, mean of per-seed crossings); at $n = 40$ seed 1 does not cross by $m = 96$, which was cut at 48 000 steps.

✓ **Finding**

- **Given time, a threshold appears.** At $n = 32$ both seeds at $m = 64$ generalize (exact 0.995 / 0.897), with train ≈ test.
- **It sits at 2–2.5× the adjacency-row width** (48 vs 22 at $n = 32$; ≥ 72 vs 30 at $n = 40$), and the adjacency rows had a quarter of the steps — which do not lower their critical width (Q3 step budget).
- **Seeds split.** After the first plateau, one seed often stalls on a partial solution for 50 000+ steps (pair 0.79–0.82) while the other solves the task; adjacency-row seeds agree within a few width units.

! **Caveat**

Two sizes, and $n = 40$ is a lower bound, so there is no growth exponent for edge tokens; the cap also cut the widest runs short. Extending it to $n = 48$ / 56 would need ≥ 2 sessions per size.

13. Encodings — precomputed walks

Two ways to hand the model part of the walk, each appended to the adjacency row, on the q3lr protocol (128k graphs, 24k steps, LR 10^{-3}, 2 seeds) at $n = 32$–56, with $n = 72$ added for k-hop rows, whose widths stay small (147 runs):

- ***k*-hop rows** — each token also gets its k-hop reachability row $[(A+I)^k > 0]$: k hops for free, leaving connectivity at about diameter $/k$ on the same graphs. No answer is given away: k-hop reachability alone gets 0 % of the graphs exactly right for $k \leq 4$ (shortcut audit).
- **RWSE** — return probabilities $\mathrm{diag}(P^j)$, $j = 1...K$, $P = D^{-1}(A+I)$, the structural encoding of GraphGPS-style models. Audit first: gradient boosting on these features alone predicts whether the graph is connected at chance for $K \leq 8$, but at 0.90 / 0.72 ($n = 40$ / 56) for $K = 16$ — the usual default — and 0.997 / 0.98 for $K = 32$: long walks settle at deg / vol(component), which gives the split away. $K = 8$ is the real encoding; $K = 16$ runs as a positive control.

input	$n = 32$	40	48	56	72	growth (90 %)
adjacency rows (Q3)	21.9	30.1	42.3	57.9	–	$n^{1.74}$ (1.59–1.90)
+ RWSE, $j \leq 8$	23.3	31.9	45.2	57.9	–	$n^{1.65}$ (1.46–1.84)
+ RWSE, $j \leq 16$ (leaks)	–	23.5	–	48.2	–	–
+ 2-hop rows	**9.2**	**13.0**	**16.7**	**18.3**	**26.7**	$n^{1.27}$ (1.05–1.51)
+ 4-hop rows	3.5†	**3.6**	**3.7**	**4.5**	**6.5**	$n^{1.03}$ (0.79–1.25)

Table 18: Critical width by input encoding (test pair ≥ 0.95, final epoch, mean of per-seed crossings); growth: least-squares slope over the sizes shown, with a 90 % seed bootstrap. † one seed only (the $n = 32$ data costs 4 h to build); not in the fit.

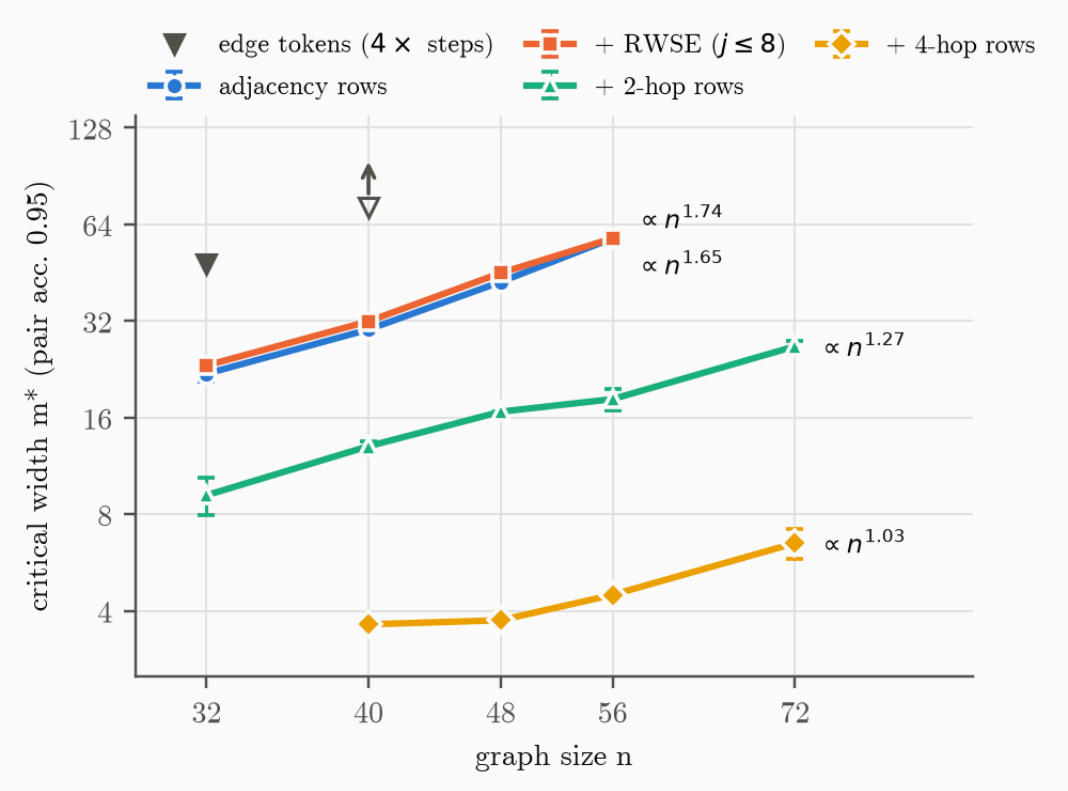


Figure 8: Critical width vs n per encoding (log–log; bars: the two seeds). Edge tokens had 4× the steps; at $n = 40$ only a lower bound.

✓ **Finding**

- **Free hops cut the width and its growth.** Two hops cut the critical width 2.3–3.2× and the growth from $n^{1.74}$ to $n^{1.27}$; four hops leave width 4–7 growing as $n^{1.03}$. The exponent falls with every hop handed over, and the bootstrap ranges of adjacency and 2-hop rows do not overlap.
- **Not linear.** The two-point estimate of 10-15 ($n^{1.00}$ for 2-hop rows) came from the flat stretch between $n = 48$ and 56; over five sizes the slope is 1.27, its range only just reaching 1.
- **RWSE adds nothing**: with $K = 8$ the critical width matches adjacency rows at every size.
- **A leaked shortcut lowers the measured width where it leaks**: RWSE-16 saves 22 % at $n = 40$ (audit 0.90) and 17 % at $n = 56$ (0.72).
- Fitting and generalizing coincide for every encoding (within ≈ 2).

◆ **What it means**

The width a transformer needs to learn connectivity tracks the number of hops it must still compute, more than the number of nodes. On `swap` the diameter grows with n (8.6 → 13.2 over $n = 32$–72), so part of Q3′s superlinear growth is path length — the question Q7b would have asked, answered here on the same graphs. Even with half the hops given away the growth stays above linear (the rate Yehudai et al.'s constant-depth construction allows); with three quarters given away it is about linear. Input encodings divide into those that do part of the search (k-hop rows), those that are only harder to read (edge tokens, projected IDs), and those that add nothing (RWSE) — unless, like RWSE-16, they leak the answer.

! **Caveat**

Four or five sizes spanning ≈ 2× in n, 2 seeds; local slopes between neighbours vary (2-hop rows: 1.6 / 1.4 / 0.6 / 1.5). Below width 16 the model has one attention head (head size 8), so the 4-hop widths (3–7) sit near the smallest model that works at all, on a coarse grid (2, 4, 6, 8 …).

14. Probing — what does each layer compute?

The scaling results say **how much** width connectivity needs, not **why**. Two mechanisms would explain a width growing with n: (a) **path doubling** — each layer squares the reach ($A \to A^2 \to A^4$...), so after layer ℓ a node knows its 2^ℓ-hop neighbourhood; (b) **storing the reachable set** — each node's vector holds the n bits "which nodes I reach", which needs width growing with n. A narrow model should then stall at some layer.

Set-up. The `probe` sweep retrains the final protocol (128 000 graphs, depth 6, rate 10^{-3}, 24 000 steps) at $n = 40$ ($m = 16$, 24, 32, 48) and $n = 56$ ($m = 32$, 48, 64, 96), 2 seeds, and saves the weights — widths on both sides of each threshold (30.1 / 57.9). On 500 fresh `swap` graphs (300 held out) we read the residual stream after the read-in (layer 0) and after every block, layer-normed, and fit two linear probes per layer for "distance $d(i, j) \leq k$", $k \in \{1, 2, 3, 4, 6, 8, \infty\}$ (∞ = connected), scored by held-out ROC-AUC:

- **pair probe** $h_i^\top W h_j$ — the form of the model's own read-out;
- **set probe** $h_i \to \mathbb{R}^n$ — node i's whole k-hop set from its own vector.

Every probe is also fitted on the **untrained** network (same initial weights): a bilinear form on adjacency rows already computes 2-hop counts, so only what the trained model adds counts (`probe_layers.py`).

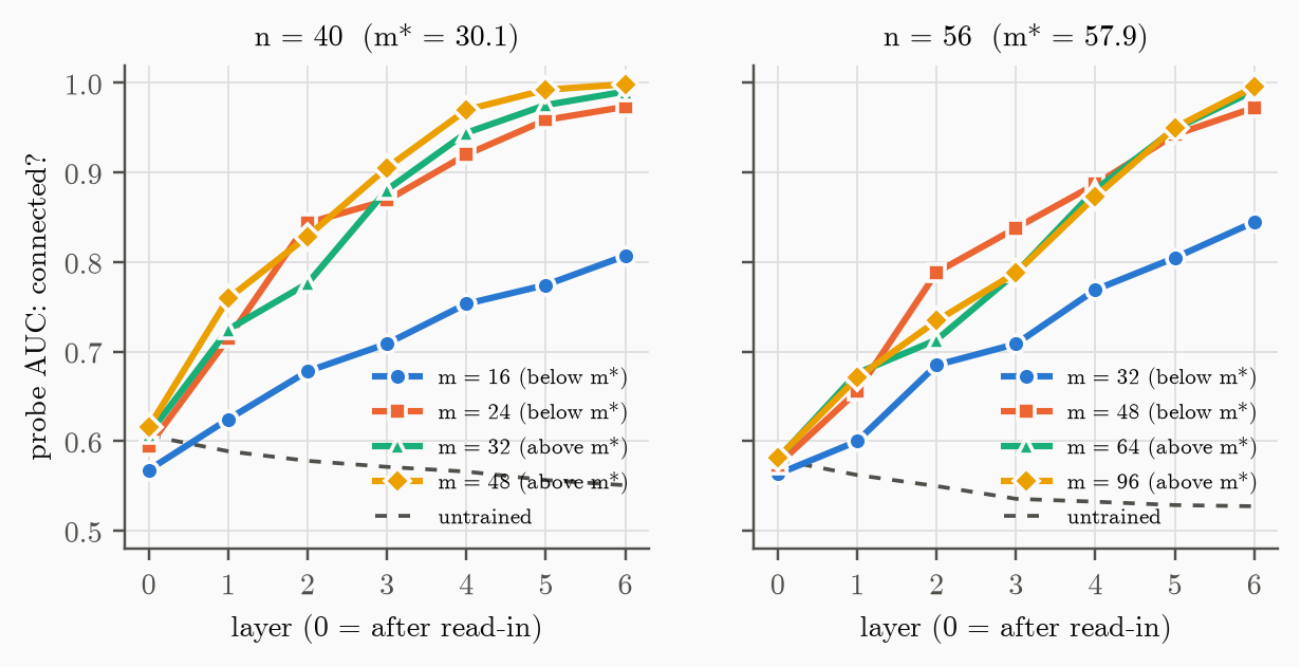


Figure 9: Pair-probe AUC for "connected" after each layer (mean of 2 seeds); dashed: untrained networks. The model's own pair accuracy: $n = 40$: 0.79 / 0.92 / 0.96 / 0.99, $n = 56$: 0.82 / 0.93 / 0.97 / 0.98 for the four widths.

pair probe, $n = 40$, $m = 48$	L0	L1	L2	L3	L4	L5	L6
$k = 1$	1.00	0.96	0.96	0.96	0.95	0.94	0.92
$k = 3$	0.78	0.92	0.93	0.94	0.94	0.93	0.92
$k = \infty$ (connected)	0.62	0.76	0.83	0.91	0.97	0.99	**1.00**
untrained, $k = \infty$	0.62	0.60	0.58	0.57	0.57	0.57	0.56

Table 19: Pair-probe AUC by layer and distance (mean of 2 seeds).

✓ Finding

- **"Connected" builds up gradually.** The connected signal rises by a similar step at every layer and reaches ≥ 0.95 only at layer 4–6, at both sizes. There is no single layer where it appears.
- **No hop hierarchy.** 3-hop information jumps at layer 1 (0.73–0.79 → 0.86–0.92) and then stays flat; 1-hop detail **falls** (1.00 → 0.85–0.95 at the last layer). Neither path doubling (2^ℓ hops) nor one hop per layer shows up in what a linear probe can read.
- **Nodes do not hold their reachable set.** From one node's vector, its connected set decodes at AUC 0.60–0.89 after the last layer, and 1-hop at 0.67–0.89 — **below** the untrained network's 0.88–1.00. Comparing two nodes' vectors gives 0.99–1.00.
- **Width.** Models far below the critical width ($n = 40$, $m = 16$; $n = 56$, $m = 32$) keep pace for a layer or two, then gain less per layer and end at 0.81 / 0.84. Models just below it ($m = 24$ / 48) look like the wide ones on this probe (0.97) — the gap between them and m^* is in the last few percent the probe does not resolve.
- Narrow read-ins lose neighbour detail: at $n = 40$, $m = 16$ even layer 0 decodes 1-hop at only 0.91.

◆ What it means

The model does not keep a table of distances and does not store, per node, the list of nodes it reaches. It seems to give each node a signature of its component that two nodes can compare — refined a little in every layer. Width sets how much each layer refines: with six layers, a model far below the critical width runs out of layers. This argues against hypothesis (b) as the source of the n^2; the cost of the signature has to be found another way.

! Caveat

Linear probes show what is **easily readable**, not what the model uses; a nonlinear probe could find more. Two sizes, 2 seeds each.

15. What it all means so far

1. **Measuring width is a protocol problem first.** Learning rate and training-set size both interact with width strongly enough to fake or hide a width effect. Every later sweep tunes the rate per width and checks the data ceiling.
2. **Benchmarks decide the answer.** The same architecture needs width 8 on the leaky task and $\geq$ 32–64 on the audited one at similar n. A width study is only as good as its shortcut audit — a methodological contribution in its own right.
3. **Likely new.** A novelty check (`docs/novelty-check-2026-10-08.md`) found no prior work measuring how the width needed to **learn** connectivity scales with n. The closest — Yehudai et al.'s critical width, linear in n — measures **fitting** on counting tasks at depth 1 with 5 000 graphs and a fixed learning-rate grid, exactly the confounds Q1 and Q1c expose.
4. **Representation is not learning.** Theory says width $\approx n$ at constant depth (Yehudai et al.), or width just above $\sqrt{N}$ at depth $\log N$ (Sanford et al.), is **enough to represent** connectivity. In practice the width needed to **learn** it grows roughly as $n^{1.7}$ over $n = 32$–56 — with 128 000 graphs, with twice the training steps, with the learning rate tuned, and with a fixed-width input encoding — and the data has to grow too; at $n = 128$ nothing up to $m = 128$ learns. At depth 2, where the linear-width construction lives, it grows faster still ($\approx n^3$), and with Sanford et al.'s own encoding, even with all-pairs labels, the width that learns is ≥ 48 at $n = 32$ — $\geq 5\times$ their $\sqrt{N}$. This matches Saparov et al. (ICLR 2025): transformers struggle to learn search even when it is representable.
5. **A hypothesis for the n^2.** n^2 is the number of bits in the adjacency matrix, and Yehudai et al. note that quadratic width suffices for **any** graph task by copying the whole graph into every token. SGD may be finding that brute-force solution rather than the efficient log-depth one. Q6 rules out the **input** width, not this. Probing (10-21) finds no path doubling and no per-node reachable set: the model builds a component signature gradually over the layers.
6. **Path length is a large part of it.** Precomputing two hops lowers the growth from $n^{1.74}$ to $n^{1.27}$, four hops to $n^{1.03}$ (Encodings — precomputed walks); the width tracks the hops left to compute.

! Caveat

- All slopes come from four graph sizes spanning less than a factor of two in n, with 2–3 seeds; local slopes between neighbouring sizes range from ≈ 1.2 to ≈ 2.9.
- The learning rate is 10^{-3} at all four sizes since 10-20; it was tuned (against $3 \cdot 10^{-3}$) at $n = 40$ / 56 only.
- Exact-match gets stricter as n^2 grows; pair-accuracy thresholds are reported alongside from now on.
- The `swap` diameter grows with $\log n$, so n and path length rise together; Q7b is dropped (supervisor, 10-13), but the k-hop encodings separate them on the same graphs.
- One learning rate in the fine grid ($3 \cdot 10^{-3}$), for budget reasons; it is too high at $m = 128$, which is left out of the fit.

16. Next steps

Item	Purpose	Status
Edge tokens, 4× steps	`enc_edges96k`: does a critical width appear with time?	done: 2–2.5× adjacency rows

Walk encodings	k-hop rows ($k = 2$, 4), RWSE ($j \le 8$; $j \le 16$ leaks)	done: growth $n^{1.74} \to n^{1.27} \to n^{1.03}$ with 0 / 2 / 4 free hops
Thesis write-up	department LaTeX template; chapters from this report	ch. 1–3 done; ch. 4 next
Probing	read what each layer computes: path doubling (A^{2^ℓ}) or a copy of the whole graph?	done: gradual, pairwise; no doubling
Q7b diameter	fix n, vary the diameter	dropped (supervisor)
Q4 Graphormer bias	shortest-path bias gives connectivity away; needs a task it doesn't leak	planned
Q2 task hierarchy	retrieval < connectivity < shortest path, each with its own audit	planned

Table 20: Week 6 is the first progress review (Явц 1); research freeze in week 11.

Reproduce. Every number above comes from `python width_sweep.py <sweep> --analyze` or `--curves` over `results/width/<sweep>*.jsonl` (sweeps `q1`, `q1b`, `q1c`, `q3pilot`, `q3pilot2`, `q3probe`, `q3trim`, `q3fine`, `q3data`, `q3big`, `q3steps`, `q3lr`, `q6proj`, `q7depth`, `q7c`, `enc_edges`, `enc_edges96k`, `enc_khop2`, `enc_khop4`, `enc_rwse8`, `enc_rwse16`, `probe`) and `--scaling` for critical widths; probes from `python probe_layers.py --summary` the audit from `python audit_width_data.py` figures from `reports/figures/width_progress_plots.py`. The Sanford section: `python sanford_repro.py <sweep> --analyze` over `results/sanford/` (`a1k`, `a100k_lr1e4`, `bwidth`, `b2`, `b2long`) and `--audit`.